

Implementação e Análise de Desempenho de Multiplicação de Matrizes em CPU, OpenMP e CUDA*

Davi Cândido de Almeida¹
Vitor Leite Setraghi²

Resumo

Este artigo apresenta a implementação e a análise de desempenho de diferentes versões de multiplicação de matrizes quadradas, considerando abordagens sequenciais em CPU, paralelização com OpenMP e execução em GPU com CUDA. A operação avaliada corresponde ao padrão GEMM, amplamente utilizado em rotinas de álgebra linear e bibliotecas como BLAS (NETLIB, 2026). Foram desenvolvidas versões naive, com matriz transposta, com blocagem, com OpenMP, com blocagem associada a OpenMP, além de kernels CUDA naive e CUDA com tiled. A avaliação foi realizada a partir da medição do tempo de execução, do cálculo de desempenho em GFLOP/s e da verificação do erro absoluto máximo em relação a uma implementação de referência. As versões paralelas em CPU utilizaram diretivas OpenMP (OPENMP ARCHITECTURE REVIEW BOARD, 2021), enquanto as versões em GPU seguiram o modelo de programação CUDA, baseado em grids, blocos e threads (NVIDIA CORPORATION, 2026). Os resultados permitiram comparar o impacto das otimizações de acesso à memória, do paralelismo em CPU e do uso de memória compartilhada na GPU.

Palavras-chave: GEMM; multiplicação de matrizes; OpenMP; CUDA; desempenho; GPU.

* Artigo apresentado ao Instituto de Ciências Exatas e Informática da Pontifícia Universidade Católica de Minas Gerais como parte da entrega do Trabalho Prático 3 da disciplina de Arquitetura de Computadores.

¹ Aluno do curso de Ciência da Computação – davikandido@gmail.com.

² Aluno do curso de Ciência da Computação – vitorsetraghi@gmail.com.

1 INTRODUÇÃO

A multiplicação de matrizes é uma das operações mais importantes em computação científica, aprendizado de máquina, computação gráfica e processamento numérico. Em sua forma geral, a operação GEMM, do inglês *General Matrix-Matrix Multiplication*, calcula a matriz C a partir do produto entre duas matrizes A e B (NETLIB, 2026). Para matrizes quadradas de dimensão N , cada elemento de C é obtido por meio de um produto interno entre uma linha de A e uma coluna de B .

Embora a implementação conceitual da multiplicação de matrizes seja simples, seu desempenho depende fortemente da forma como os dados são acessados na memória e de como o paralelismo do hardware é explorado. A versão ingênua, com três laços aninhados, realiza $O(N^3)$ operações e pode desperdiçar desempenho por acessar a segunda matriz com baixa localidade espacial. Por isso, técnicas como transposição, blocagem, vetorização, paralelismo com OpenMP e uso de GPU com CUDA são alternativas importantes para reduzir o tempo de execução.

O objetivo deste trabalho foi implementar e comparar diferentes versões do GEMM, avaliando o impacto de cada otimização no desempenho. As versões implementadas foram: sequencial ingênua, sequencial com matriz transposta, sequencial com blocagem, paralela com OpenMP, paralela com OpenMP e blocagem, CUDA ingênua e CUDA com *tiling*. Todas as versões foram validadas por comparação com uma versão sequencial de referência, utilizando o erro absoluto máximo como critério de corretude.

Além da implementação, o trabalho exigiu a medição experimental dos tempos, o cálculo de GFLOP/s, a análise de *speedup*, a análise de eficiência no caso OpenMP e a discussão crítica dos resultados. Assim, o relatório busca não apenas apresentar os tempos obtidos, mas também explicar por que determinadas versões foram mais rápidas ou mais lentas em função do acesso à memória, do tamanho das matrizes, do custo de paralelização e do custo de transferência entre CPU e GPU.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Multiplicação de matrizes e custo computacional

A multiplicação de duas matrizes quadradas A e B , ambas de dimensão $N \times N$, produz uma matriz C , também de dimensão $N \times N$. Cada elemento C_{ij} é calculado pela Equação 1.

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} \cdot B_{kj} \quad (1)$$

Para cada elemento de C , são feitas N multiplicações e N somas aproximadas. Con-

siderando a métrica usual de desempenho para GEMM, o número total de operações de ponto flutuante é aproximado por $2N^3$. Por isso, o desempenho em GFLOP/s foi calculado pela Equação 2.

$$GFLOP/s = \frac{2N^3}{tempo_s \cdot 10^9} \quad (2)$$

O *speedup* foi calculado comparando o tempo de uma versão de referência com o tempo da versão otimizada, como mostra a Equação 3. No caso das versões OpenMP, também foi calculada a eficiência, dividindo o *speedup* pelo número de *threads*, conforme a Equação 4.

$$Speedup = \frac{T_{referencia}}{T_{otimizado}} \quad (3)$$

$$Eficiencia = \frac{Speedup}{numero\ de\ threads} \quad (4)$$

2.2 Localidade de memória

Em matrizes armazenadas em ordem de linha, como ocorre em C, elementos consecutivos de uma mesma linha ficam próximos na memória. Na versão ingênua com laços i , j e k , a matriz A é acessada por linha, mas a matriz B é acessada por coluna. Esse acesso por coluna prejudica a localidade espacial, pois os elementos usados em sequência não estão necessariamente próximos na memória.

A versão com transposição busca reduzir esse problema. Antes da multiplicação, a matriz B é transposta para uma nova matriz BT . Assim, o cálculo de C_{ij} passa a acessar uma linha de A e uma linha de BT , melhorando a localidade dos acessos. A versão com blocagem, por sua vez, divide a multiplicação em blocos menores, tentando reutilizar dados em cache antes que eles sejam substituídos.

2.3 OpenMP e paralelismo em CPU

OpenMP é uma interface de programação usada para paralelizar trechos de código em arquiteturas de memória compartilhada (OPENMP ARCHITECTURE REVIEW BOARD, 2021). Neste trabalho, a paralelização foi aplicada principalmente aos laços externos da multiplicação de matrizes por meio da diretiva `#pragma omp parallel for`. A ideia é distribuir o cálculo de diferentes posições ou blocos da matriz C entre várias *threads*.

O ganho de desempenho com OpenMP depende da quantidade de núcleos disponíveis, do custo de criação e sincronização das *threads*, do tamanho do problema e da forma como os dados são acessados. Em problemas pequenos, o custo da paralelização pode ser maior que o ganho. Além disso, se o ambiente possuir poucos núcleos, aumentar muito o número de *threads*

pode causar sobrecarga e até piorar o tempo de execução.

2.4 CUDA e paralelismo em GPU

CUDA permite executar funções chamadas *kernels* em GPUs NVIDIA (NVIDIA CORPORATION, 2026). Em uma implementação CUDA de GEMM, cada *thread* pode ser responsável por calcular um elemento da matriz C . A versão direta implementa essa ideia de forma simples: cada *thread* calcula uma célula percorrendo a dimensão k .

A versão *tiled* utiliza memória compartilhada para carregar pequenos blocos das matrizes A e B dentro de cada bloco CUDA. Essa técnica reduz acessos repetidos à memória global, pois os dados carregados em memória compartilhada podem ser reutilizados por várias *threads* do mesmo bloco. Em geral, esse tipo de otimização é mais eficiente para matrizes maiores, nas quais o custo computacional domina o custo de inicialização e transferência.

3 METODOLOGIA EXPERIMENTAL

Os experimentos foram organizados em duas partes principais. A primeira avaliou as versões em CPU, implementadas em C e compiladas com `gcc`. A segunda avaliou as versões em GPU, implementadas em CUDA e compiladas com `nvcc`. Em todas as execuções, cada configuração foi repetida cinco vezes, e o tempo médio foi utilizado para o cálculo de GFLOP/s.

3.1 Ambiente de execução

O ambiente utilizado nos testes foi uma instância com processador Intel Xeon e GPU NVIDIA Tesla T4. A Figura 1 mostra as informações da CPU, incluindo arquitetura `x86_64`, dois processadores lógicos disponíveis e caches L1, L2 e L3. A Figura 2 mostra as informações da GPU, com placa Tesla T4, memória de 15360 MiB e versão CUDA 13.0 disponível no ambiente.

```
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 46 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 2
On-line CPU(s) list: 0,1
Vendor ID: GenuineIntel
Model name: Intel(R) Xeon(R) CPU @ 2.00GHz
CPU family: 6
Model: 85
Thread(s) per core: 2
Core(s) per socket: 1
Socket(s): 1
Stepping: 3
BogoMIPS: 4000.36
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36
Hypervisor vendor: KVM
Virtualization type: full
L1d cache: 32 KiB (1 instance)
L1i cache: 32 KiB (1 instance)
L2 cache: 1 MiB (1 instance)
L3 cache: 38.5 MiB (1 instance)
NUMA node(s): 1
NUMA node0 CPU(s): 0,1
Vulnerability Gather data sampling: Not affected
Vulnerability Indirect target selection: Vulnerable
Vulnerability Itlb multihit: Not affected
Vulnerability L1tf: Mitigation; PTE Inversion
Vulnerability Mds: Vulnerable; SMT Host state unknown
Vulnerability Meltdown: Vulnerable
```

Figura 1 – Informações da CPU utilizada nos experimentos.

```

Thu Jun 25 19:29:50 2026
+-----+
| NVIDIA-SMI 580.82.07                  Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----+-----+-----+
| GPU  Name            Persistence-M   Bus-Id        Disp.A    Volatile Uncorr. ECC  |
| Fan  Temp            Pwr            Usage      Memory      GPU-Util    Compute M.  |
|              Perf            Pwr:Usage/Cap      Usage      MIG M.      |
+-----+-----+-----+
|    0  Tesla T4             Off          00000000:00:04:0  Off          0          0          |
| N/A   42C             12W /   70W      0MiB /  15360MiB  0%         Default  |
|                               N/A                               |
+-----+-----+-----+
+-----+
| Processes:                              |
| GPU   GI    CI          PID    Type    Process name                        GPU Memory |
|   ID   ID                               Name                                Usage     |
+-----+-----+-----+
| No running processes found              |
+-----+

```

Figura 2 – Informações da GPU utilizada nos experimentos.

Tabela 1 – Resumo do ambiente experimental

Item	Configuração observada
CPU	Intel Xeon CPU @ 2.00 GHz, arquitetura x86_64, 2 CPUs lógicas disponíveis
Caches da CPU	L1d 32 KiB, L1i 32 KiB, L2 1 MiB, L3 38,5 MiB
GPU	NVIDIA Tesla T4
Memória da GPU	15360 MiB
CUDA	CUDA 13.0 no ambiente de execução
Repetições	5 execuções por configuração
Métrica principal	Tempo médio e GFLOP/s
Validação	Erro absoluto máximo em relação à versão sequencial de referência

3.2 Versões implementadas

A Tabela 2 resume as versões implementadas e a principal ideia de cada uma.

Tabela 2 – Versões de GEMM implementadas

Versão	Descrição
naive	Implementação sequencial com três laços aninhados, usada também como referência de corretude.
transposed	Transpõe a matriz B antes da multiplicação, melhorando o acesso sequencial à memória.
blocked	Divide a multiplicação em blocos, avaliando tamanhos de bloco 16, 32 e 64.
openmp	Paraleliza a versão direta com OpenMP usando 1, 2, 4 e 8 <i>threads</i> .
blocked_openmp	Combina blocagem com OpenMP, usando bloco 32 e 1, 2, 4 e 8 <i>threads</i> .
cuda_naive	Kernel CUDA direto, em que cada <i>thread</i> calcula um elemento da matriz C .
cuda_tiled	Kernel CUDA com uso de memória compartilhada e blocos 16×16 .

Para a CPU, foram testados os tamanhos $N = 128$, $N = 256$ e $N = 512$. Para a GPU, também foi incluído $N = 1024$, pois a GPU suporta melhor problemas maiores e permite observar com mais clareza o ganho da versão *tiled*.

3.3 Compilação e vetorização

A versão CPU foi compilada com otimizações do compilador, usando `-O3`, `-march=native`, `-fopenmp` e relatório de vetorização. A saída de compilação indicou que alguns laços foram vetorizados com vetores de 32 bytes e 16 bytes, como mostrado na Figura 3. As mensagens de aviso relacionadas a `posix_memalign` indicam apenas que o valor de retorno da função não foi tratado no código, não impedindo a compilação nem a execução dos testes.

```

gemm_cpu.c: In function 'main':
gemm_cpu.c:166:5: warning: ignoring return value of 'posix_memalign' declared with attribute 'warn_unused_result'
166 |     posix_memalign((void**)&A, ALIGNMENT, sizeof(float)*N*N);
    |     ~~~~~
gemm_cpu.c:167:5: warning: ignoring return value of 'posix_memalign' declared with attribute 'warn_unused_result'
167 |     posix_memalign((void**)&B, ALIGNMENT, sizeof(float)*N*N);
    |     ~~~~~
gemm_cpu.c:168:5: warning: ignoring return value of 'posix_memalign' declared with attribute 'warn_unused_result'
168 |     posix_memalign((void**)&BT, ALIGNMENT, sizeof(float)*N*N);
    |     ~~~~~
gemm_cpu.c:169:5: warning: ignoring return value of 'posix_memalign' declared with attribute 'warn_unused_result'
169 |     posix_memalign((void**)&C, ALIGNMENT, sizeof(float)*N*N);
    |     ~~~~~
gemm_cpu.c:170:5: warning: ignoring return value of 'posix_memalign' declared with attribute 'warn_unused_result'
170 |     posix_memalign((void**)&Cref, ALIGNMENT, sizeof(float)*N*N);
    |     ~~~~~
gemm_cpu.c:137:44: optimized: loop vectorized using 32 byte vectors
gemm_cpu.c:137:44: optimized: loop versioned for vectorization because of possible aliasing
gemm_cpu.c:137:44: optimized: loop vectorized using 16 byte vectors
gemm_cpu.c:66:31: optimized: loop vectorized using 32 byte vectors
gemm_cpu.c:66:31: optimized: loop vectorized using 16 byte vectors
gemm_cpu.c:94:44: optimized: loop vectorized using 32 byte vectors
gemm_cpu.c:94:44: optimized: loop versioned for vectorization because of possible aliasing
gemm_cpu.c:94:44: optimized: loop vectorized using 16 byte vectors
gemm_cpu.c:105:13: optimized: basic block part vectorized using 32 byte vectors
gemm_cpu.c:118:13: optimized: basic block part vectorized using 32 byte vectors
gemm_cpu.c:118:13: optimized: basic block part vectorized using 32 byte vectors
gemm_cpu.c:66:31: optimized: loop vectorized using 32 byte vectors
gemm_cpu.c:66:31: optimized: loop vectorized using 16 byte vectors
gemm_cpu.c:118:13: optimized: basic block part vectorized using 32 byte vectors
gemm_cpu.c:118:13: optimized: basic block part vectorized using 32 byte vectors

```

Figura 3 – Relatório de compilação e vetorização da versão CPU.

4 IMPLEMENTAÇÃO

4.1 Versão sequencial ingênua

A versão ingênua utiliza três laços aninhados. O laço externo percorre as linhas da matriz A , o segundo percorre as colunas da matriz B , e o terceiro realiza o produto interno. Essa versão é simples e serve como base para validação das demais versões, mas não utiliza de forma ideal a hierarquia de memória.

4.2 Versão com matriz transposta

Na versão com matriz transposta, a matriz B é convertida para BT antes da multiplicação. Com isso, o acesso que antes percorria colunas de B passa a percorrer linhas de BT . Essa alteração melhora a localidade espacial e reduz parte do custo de acesso à memória, principalmente para matrizes maiores.

4.3 Versão com blocagem

A versão com blocagem percorre a multiplicação em submatrizes menores. Foram avaliados blocos de tamanho 16, 32 e 64. A ideia é aumentar o reuso dos dados carregados em cache, evitando que linhas e colunas usadas repetidamente sejam removidas antes de serem

reaproveitadas.

4.4 Versões com OpenMP

A versão OpenMP aplica paralelismo aos laços externos usando `parallel for` com `collapse(2)` e escalonamento estático. Também foi implementada uma versão combinando OpenMP com blocagem. Foram avaliadas configurações com 1, 2, 4 e 8 *threads*. Como o ambiente apresentou apenas dois processadores lógicos disponíveis, os resultados com 4 e 8 *threads* devem ser interpretados considerando o efeito de sobrecarga e sobreassinatura.

4.5 Versões CUDA

Na versão CUDA direta, cada *thread* calcula um elemento de C . O índice da linha e da coluna é calculado a partir de `blockIdx`, `blockDim` e `threadIdx`. Essa versão é simples, mas realiza muitos acessos à memória global.

Na versão CUDA *tiled*, blocos de A e B são carregados para memória compartilhada. Cada bloco CUDA utiliza *tiles* de tamanho 16×16 . Após o carregamento de cada *tile*, as *threads* sincronizam, realizam parte do produto interno e passam para o próximo *tile*. Essa estratégia reduz acessos redundantes à memória global e melhora o desempenho do kernel.

5 RESULTADOS E DISCUSSÃO

5.1 Resultados em CPU

As Figuras 4 e 5 apresentam a tabela de resultados obtida para as versões em CPU. As colunas registram a versão executada, o tamanho da matriz, o tamanho do bloco, o número de *threads*, o número de repetições, o tempo médio, o melhor tempo, o desempenho em GFLOP/s e o erro absoluto máximo.

	version	N	BS	threads	repeats	avg_time_s	best_time_s	GfLOPS	max_abs_error
0	naive	128	32	2	5	0.002742	0.002676	1.529654	0.000000
1	transposed	128	32	2	5	0.002919	0.002336	1.436858	0.000008
2	blocked	128	16	2	5	0.001033	0.000992	4.061778	0.000000
3	blocked	128	32	2	5	0.000624	0.000581	6.723550	0.000000
4	blocked	128	64	2	5	0.000493	0.000469	8.506722	0.000000
5	openmp	128	32	1	5	0.002912	0.002728	1.440554	0.000000
6	blocked_openmp	128	32	1	5	0.000879	0.000611	4.772601	0.000000
7	openmp	128	32	2	5	0.005789	0.001327	0.724482	0.000000
8	blocked_openmp	128	32	2	5	0.001993	0.000332	2.104072	0.000000
9	openmp	128	32	4	5	0.005767	0.002850	0.727252	0.000000
10	blocked_openmp	128	32	4	5	0.002315	0.000426	1.812038	0.000000
11	openmp	128	32	8	5	0.003951	0.002777	1.061520	0.000000
12	blocked_openmp	128	32	8	5	0.001422	0.000688	2.950002	0.000000
13	naive	256	32	2	5	0.064802	0.052606	0.517801	0.000000
14	transposed	256	32	2	5	0.032633	0.023681	1.028252	0.000023
15	blocked	256	16	2	5	0.019874	0.018443	1.688359	0.000000
16	blocked	256	32	2	5	0.004740	0.004378	7.079457	0.000000
17	blocked	256	64	2	5	0.003200	0.003056	10.484715	0.000000
18	openmp	256	32	1	5	0.041955	0.022379	0.799781	0.000000

Figura 4 – Resultados CPU: primeiras configurações testadas.

15	blocked	256	16	2	5	0.019874	0.018443	1.688359	0.000000
16	blocked	256	32	2	5	0.004740	0.004378	7.079457	0.000000
17	blocked	256	64	2	5	0.003200	0.003056	10.484715	0.000000
18	openmp	256	32	1	5	0.041955	0.022379	0.799781	0.000000
19	blocked_openmp	256	32	1	5	0.006997	0.004799	4.795465	0.000000
20	openmp	256	32	2	5	0.034203	0.023539	0.981024	0.000000
21	blocked_openmp	256	32	2	5	0.009889	0.006975	3.393179	0.000000
22	openmp	256	32	4	5	0.030360	0.022040	1.105235	0.000000
23	blocked_openmp	256	32	4	5	0.005296	0.004823	6.335609	0.000000
24	openmp	256	32	8	5	0.018757	0.016617	1.788896	0.000000
25	blocked_openmp	256	32	8	5	0.006023	0.004099	5.571473	0.000000
26	naive	512	32	2	5	0.355613	0.327268	0.754852	0.000000
27	transposed	512	32	2	5	0.166833	0.158866	1.609008	0.000046
28	blocked	512	16	2	5	0.033337	0.032465	8.052086	0.000000
29	blocked	512	32	2	5	0.024719	0.024170	10.859445	0.000000
30	blocked	512	64	2	5	0.020699	0.019549	12.968695	0.000000
31	openmp	512	32	1	5	0.180579	0.177367	1.486528	0.000000
32	blocked_openmp	512	32	1	5	0.025646	0.024600	10.466798	0.000000
33	openmp	512	32	2	5	0.127912	0.121669	2.098599	0.000000
34	blocked_openmp	512	32	2	5	0.026365	0.022000	10.181613	0.000000

Figura 5 – Resultados CPU: configurações intermediárias e finais.

A Tabela 3 resume as versões mais relevantes para cada tamanho de matriz. Para os

três tamanhos testados em CPU, a melhor versão observada foi a versão com blocagem e bloco 64. Isso mostra que, para esse ambiente, a melhoria de localidade foi mais importante do que simplesmente aumentar o número de *threads*.

Tabela 3 – Resumo dos melhores resultados em CPU

N	Referência	Tempo ref. (s)	Melhor versão	Tempo médio (s)	Speedup
128	naive	0,002742	blocked BS64	0,000493	5,56x
256	naive	0,064802	blocked BS64	0,003200	20,25x
512	naive	0,355613	blocked BS64	0,020699	17,18x

O ganho da versão com blocagem é explicado pelo melhor uso da cache. Na versão ingênua, a matriz B é acessada de forma menos favorável à localidade. Na versão bloqueada, o algoritmo trabalha com submatrizes que podem permanecer por mais tempo em níveis mais rápidos da memória, aumentando o reuso de dados. Para $N = 512$, por exemplo, a versão ingênua atingiu 0,754852 GFLOP/s, enquanto a versão bloqueada com BS64 atingiu 12,968695 GFLOP/s.

As Figuras 6, 7 e 8 apresentam os gráficos de GFLOP/s para os tamanhos testados em CPU. Em geral, as versões bloqueadas aparecem entre os melhores resultados. A versão transposta melhora em relação à ingênua em alguns casos, mas seu ganho é menor do que o obtido pela blocagem.

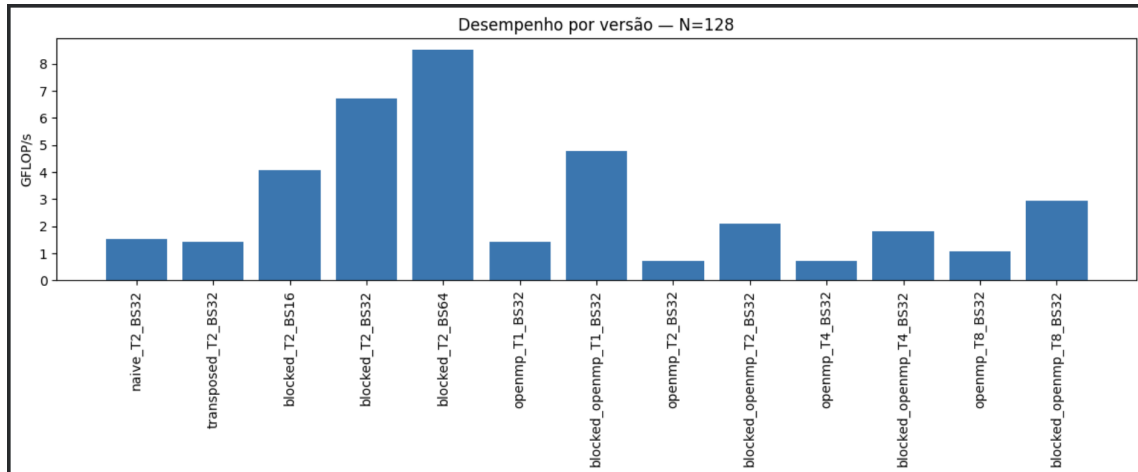


Figura 6 – Desempenho em CPU para $N = 128$.

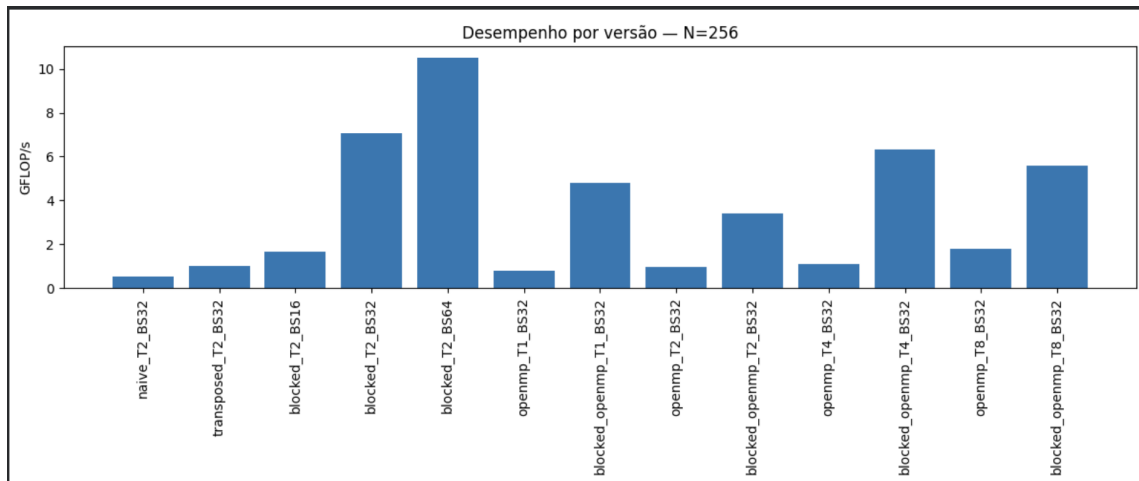


Figura 7 – Desempenho em CPU para $N = 256$.

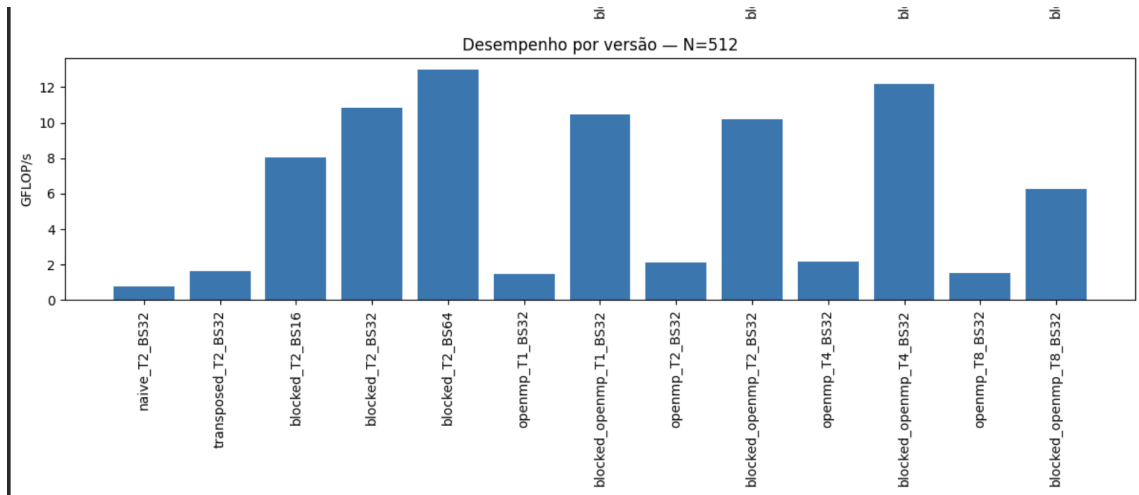


Figura 8 – Desempenho em CPU para $N = 512$.

A Figura 9 mostra o tempo médio para $N = 512$. Nessa representação, fica claro que a versão ingênua é uma das mais lentas, enquanto as versões bloqueadas reduzem significativamente o tempo de execução.

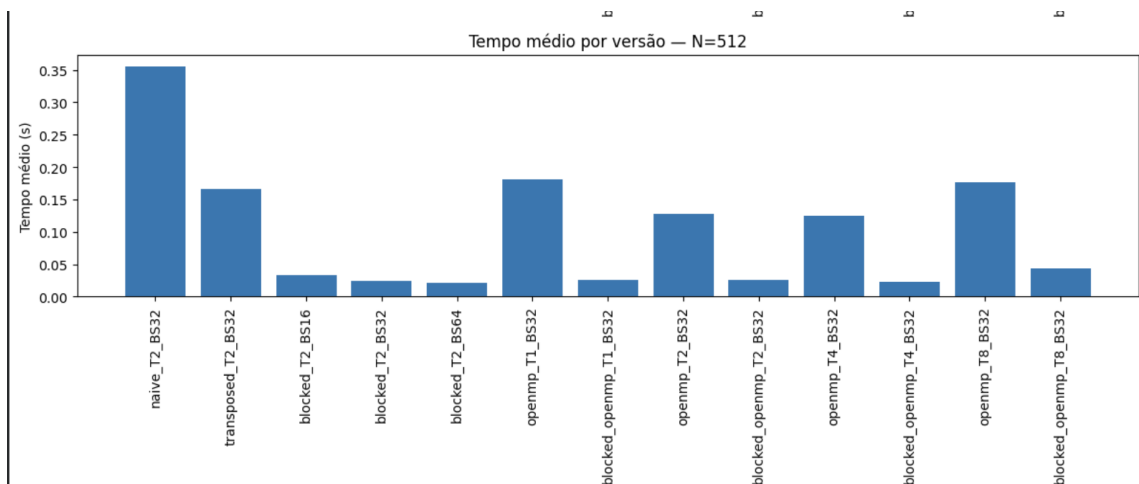


Figura 9 – Tempo médio em CPU para $N = 512$.

5.2 Speedup e eficiência com OpenMP

A Tabela 4 apresenta uma análise de *speedup* e eficiência para a versão OpenMP em $N = 512$, usando a execução com uma *thread* como referência. O objetivo dessa tabela é mostrar que o aumento do número de *threads* não gerou ganho linear no ambiente utilizado.

Tabela 4 – Speedup e eficiência da versão OpenMP para $N = 512$

Threads	Tempo médio (s)	Speedup	Eficiência
1	0,180579	1,00x	1,00
2	0,127912	1,41x	0,71
4	0,124585	1,45x	0,36
8	0,176230	1,02x	0,13

A eficiência diminui conforme o número de *threads* aumenta. Isso ocorre porque o ambiente usado disponibilizou apenas dois processadores lógicos. Assim, configurações com 4 ou 8 *threads* passam a disputar os mesmos recursos físicos, aumentando o custo de escalonamento e sincronização. Além disso, para matrizes pequenas, o custo de criar e gerenciar *threads* pode ser comparável ao tempo do próprio cálculo.

5.3 Resultados em CUDA

A Tabela 5 apresenta os resultados obtidos para as versões CUDA. O tempo do kernel foi medido separadamente dos tempos de cópia da CPU para a GPU e da GPU para a CPU. Essa separação é importante porque o kernel pode ser muito rápido, mas o tempo total da aplicação também inclui o custo de transferência de dados.

	version	N	repeats	h2d_ms	kernel_ms	d2h_ms	total_ms	kernel_GFLOPS	total_GFLOPS	max_abs_error
0	cuda_naive	128	5	0.185350	20.544046	0.044557	20.773954	0.204162	0.201902	0.000008
1	cuda_tiled	128	5	0.071744	0.089530	0.040762	0.202035	46.848236	20.760265	0.000008
2	cuda_naive	256	5	0.174835	0.104819	0.110976	0.390630	320.117228	85.898153	0.000023
3	cuda_tiled	256	5	0.165382	0.083962	0.111373	0.360717	399.640223	93.021542	0.000023
4	cuda_naive	512	5	0.618630	0.487117	0.456589	1.562336	551.069987	171.816729	0.000046
5	cuda_tiled	512	5	0.637747	0.321229	0.411712	1.370688	835.651891	195.839946	0.000046
6	cuda_naive	1024	5	1.983558	5.789254	1.431430	9.204243	370.943057	233.314539	0.000092
7	cuda_tiled	1024	5	2.244173	3.734470	1.504147	7.482790	575.043697	286.989681	0.000092

Figura 10 – Tabela de resultados das versões CUDA.

Tabela 5 – Resumo dos resultados CUDA

N	Versão	Kernel (ms)	Total (ms)	Kernel GFLOP/s	Erro máx.
128	cuda_naive	20,544046	20,773954	0,204162	0,000008
128	cuda_tiled	0,089530	0,202035	46,848236	0,000008
256	cuda_naive	0,104819	0,390630	320,117228	0,000023
256	cuda_tiled	0,083962	0,360717	399,640223	0,000023
512	cuda_naive	0,487117	1,562336	551,069987	0,000046
512	cuda_tiled	0,321229	1,370688	835,651891	0,000046
1024	cuda_naive	5,789254	9,204243	370,943057	0,000092
1024	cuda_tiled	3,734470	7,482790	575,043697	0,000092

Em todos os tamanhos, a versão `cuda_tiled` foi mais eficiente que a versão `cuda_naive`. Para $N = 512$, o kernel *tiled* reduziu o tempo de 0,487117 ms para 0,321229 ms, aumentando o desempenho de 551,07 GFLOP/s para 835,65 GFLOP/s. Para $N = 1024$, a redução do tempo de kernel foi de 5,789254 ms para 3,734470 ms.

Tabela 6 – Speedup da versão CUDA tiled em relação à CUDA naive

N	Speedup pelo tempo de kernel	Speedup pelo tempo total
128	229,48x	102,82x
256	1,25x	1,08x
512	1,52x	1,14x
1024	1,55x	1,23x

O caso de $N = 128$ apresenta um comportamento atípico na versão `cuda_naive`, com tempo de kernel muito superior ao observado nos demais tamanhos. Esse resultado provavelmente foi influenciado por overhead de primeira execução, inicialização do contexto CUDA ou variação do ambiente. Mesmo assim, ele foi mantido no relatório porque corresponde à execução registrada. Para matrizes maiores, os resultados ficam mais estáveis e mostram ganho consistente da versão *tiled*.

As Figuras 11, 12, 13 e 14 apresentam os tempos de kernel das versões CUDA. Já as Figuras 15, 16, 17 e 18 apresentam os desempenhos em GFLOP/s.

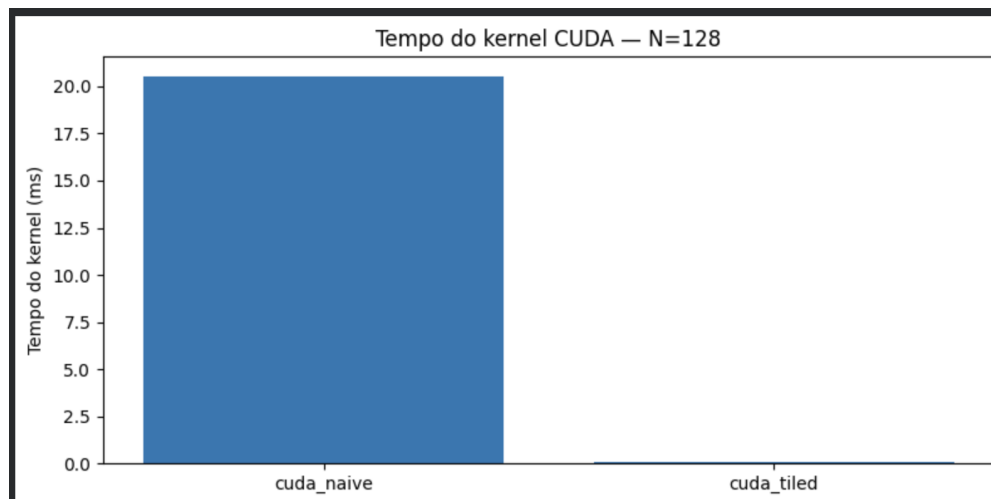


Figura 11 – Tempo do kernel CUDA para $N = 128$.

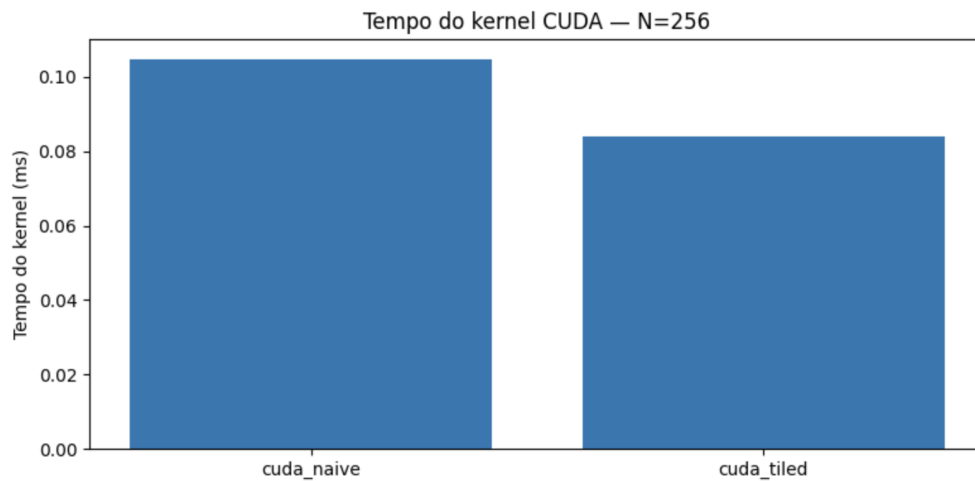


Figura 12 – Tempo do kernel CUDA para $N = 256$.

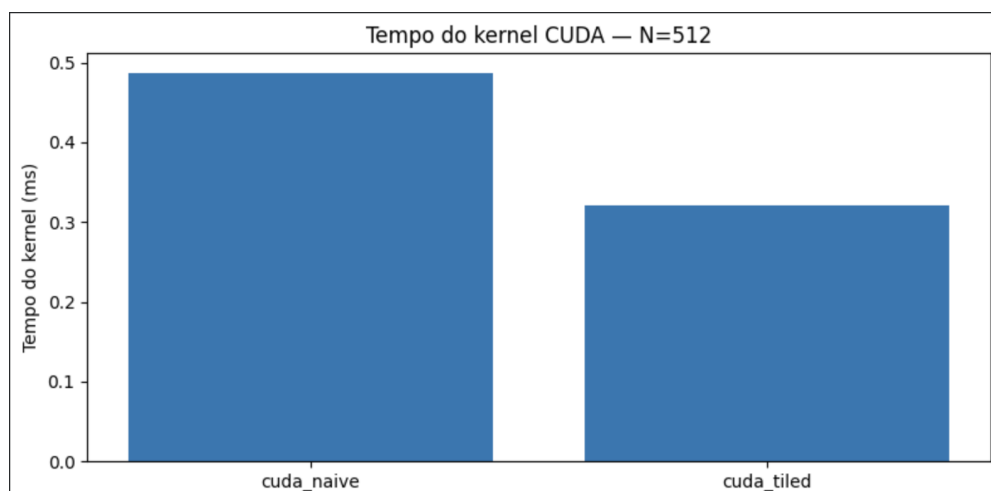


Figura 13 – Tempo do kernel CUDA para $N = 512$.

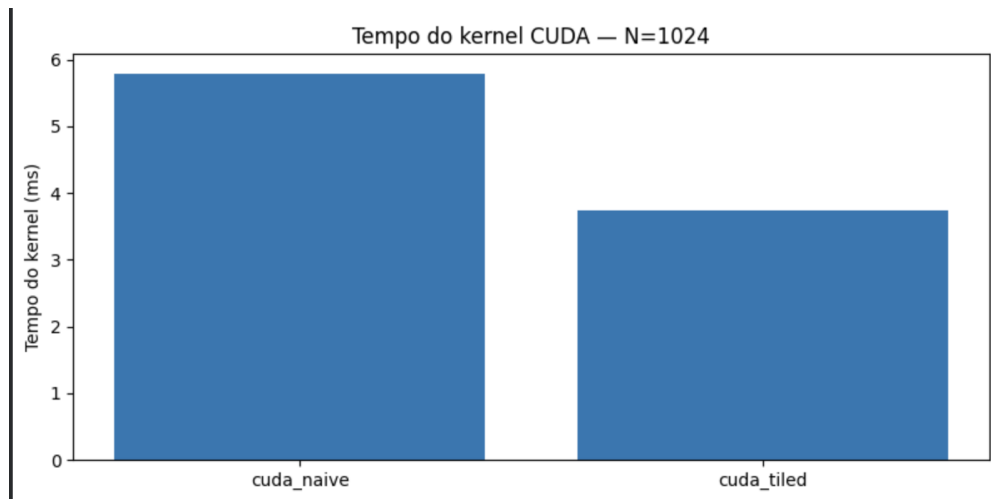


Figura 14 – Tempo do kernel CUDA para $N = 1024$.

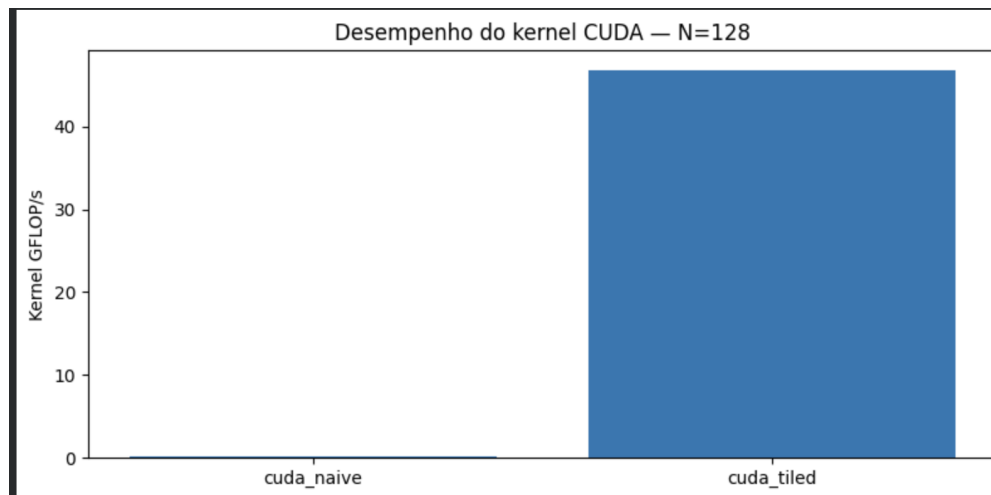


Figura 15 – Desempenho do kernel CUDA para $N = 128$.

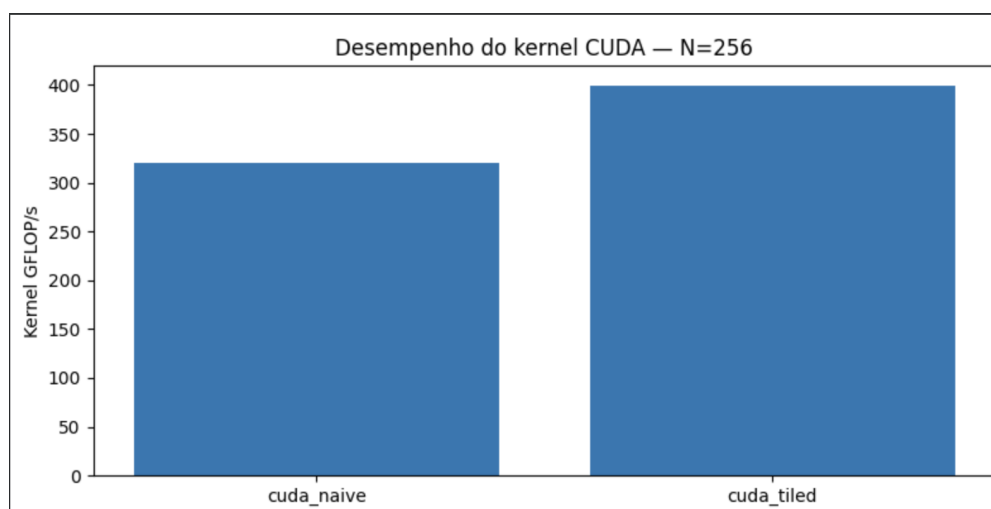


Figura 16 – Desempenho do kernel CUDA para $N = 256$.

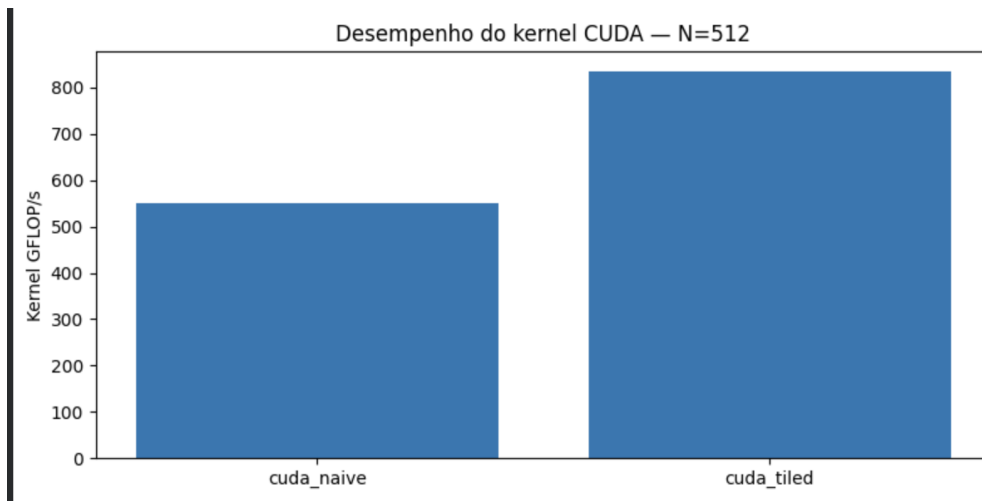


Figura 17 – Desempenho do kernel CUDA para $N = 512$.

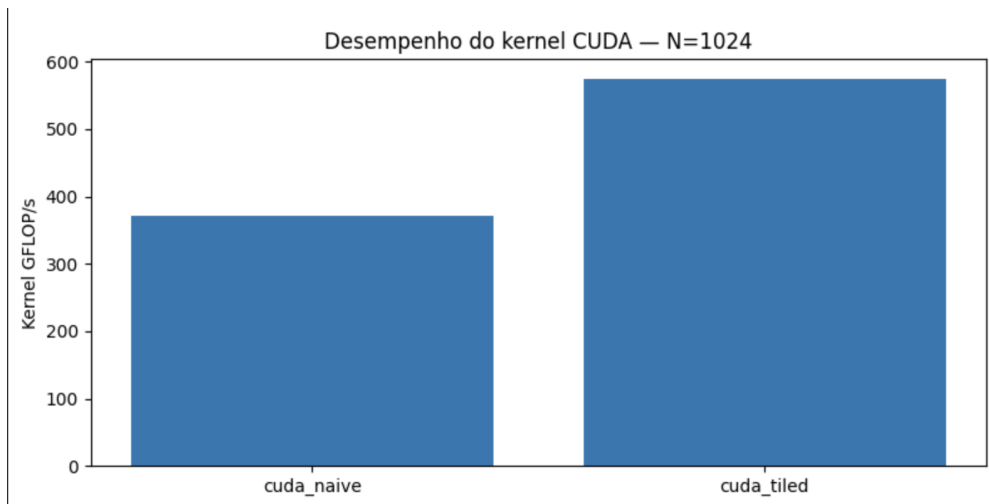


Figura 18 – Desempenho do kernel CUDA para $N = 1024$.

5.4 Comparação entre CPU e GPU

A Tabela 7 compara o melhor resultado em CPU com o melhor resultado total em GPU para os tamanhos comuns. Para tornar a comparação justa em nível de aplicação, foi utilizado o tempo total da GPU, que inclui cópia de dados para a GPU, execução do kernel e cópia do resultado de volta para a CPU.

Tabela 7 – Comparação entre melhor CPU e melhor CUDA considerando tempo total da GPU

N	Melhor CPU	Tempo CPU	Melhor CUDA total	Speedup GPU/CPU
128	blocked BS64	0,493 ms	cuda_tiled: 0,202 ms	2,44x
256	blocked BS64	3,200 ms	cuda_tiled: 0,361 ms	8,87x
512	blocked BS64	20,699 ms	cuda_tiled: 1,371 ms	15,10x

A comparação mostra que o ganho da GPU aumenta conforme o tamanho da matriz cresce. Isso ocorre porque, para matrizes pequenas, o custo fixo de transferir dados e iniciar o kernel representa uma parcela relevante do tempo total. Para matrizes maiores, o custo computacional $O(N^3)$ cresce mais rapidamente que o custo de transferência $O(N^2)$, tornando a GPU mais vantajosa.

5.5 Validação dos resultados

Todas as versões foram comparadas com uma versão sequencial de referência. Na CPU, o erro máximo foi zero na maioria das versões, com pequenas diferenças em algumas configurações devido à ordem de acumulação em ponto flutuante. Na GPU, os erros máximos variaram de 0,000008 a 0,000092. Esses valores são baixos e compatíveis com diferenças esperadas em operações de ponto flutuante em precisão simples.

6 DIFICULDADES ENCONTRADAS

Uma das principais dificuldades foi compreender que o desempenho de GEMM não depende apenas da quantidade de operações aritméticas. A forma de acesso à memória teve grande impacto nos resultados. A versão ingênua é simples, mas acessa a matriz B de maneira pouco favorável à localidade. Por isso, foi necessário implementar e testar a transposição e a blocagem para observar o efeito do uso da cache.

Outra dificuldade foi interpretar os resultados com OpenMP. A expectativa inicial era que aumentar o número de *threads* sempre reduzisse o tempo de execução. Porém, os testes mostraram que isso não ocorreu. Como o ambiente possuía poucos processadores lógicos disponíveis, o uso de 4 ou 8 *threads* gerou sobrecarga e não produziu ganho proporcional. Esse comportamento reforça que paralelizar um algoritmo não garante, por si só, melhor desempenho.

Também houve dificuldade na parte CUDA por causa da dependência do ambiente. Em uma execução inicial, o comando `nvcc` não foi encontrado, indicando que o ambiente não estava configurado com suporte CUDA. Para resolver isso, foi necessário usar o ambiente com GPU habilitada.

Na implementação CUDA, outra dificuldade foi compreender corretamente o uso de `blockIdx`, `threadIdx`, memória compartilhada e sincronização por `__syncthreads()`. A versão *tiled* exige mais cuidado do que a versão direta, pois cada bloco de *threads* precisa carregar partes de A e B , tratar limites da matriz e sincronizar antes de usar os dados compartilhados.

Por fim, também foi necessário interpretar os resultados de tempo separando kernel e transferência. A GPU pode executar o cálculo rapidamente, mas o tempo total inclui cópias

entre CPU e GPU. Essa diferença é importante para evitar conclusões incorretas sobre o desempenho real da aplicação.

7 USO DE INTELIGÊNCIA ARTIFICIAL

Ferramentas de Inteligência Artificial foram utilizadas como apoio durante o desenvolvimento e a documentação do trabalho. O uso ocorreu principalmente para esclarecer conceitos de GEMM, revisar a lógica das versões implementadas, ajudar na interpretação de erros do ambiente CUDA, organizar a estrutura do relatório e revisar a discussão dos resultados. A responsabilidade técnica pela implementação, validação e entrega permanece com os autores do trabalho.

Os prompts foram elaborados seguindo uma ideia próxima de *Spec-Driven Development*, com descrição explícita do problema, das versões que deveriam ser implementadas e das restrições do trabalho. As interações foram organizadas em quatro categorias: dúvidas conceituais, implementação, execução dos experimentos e escrita do relatório.

Tabela 8 – Registro de uso de IA no trabalho

Prompt utilizado	Finalidade	Validação humana
"Explique como funciona a multiplicação de matrizes GEMM e como calcular GFLOP/s para uma matriz $N \times N$."	Compreender a métrica de desempenho usada no relatório.	Fórmula conferida manualmente com o número de operações $2N^3$.
"Quais tópicos devo estudar para entender melhor CUDA, memória compartilhada"	Obter indicação de conteúdos e conceitos necessários para estudar programação em CUDA.	Os tópicos indicados foram usados apenas como apoio de estudo, sem substituir a implementação, os testes e a análise feita pelos autores.
"A Parte 7 deu erro <code>FileNotFoundError: ./gemm_cuda</code> . Como eu ativo a GPU no colab?"	Entender como executar CUDA precisava ser compilado antes da execução.	Célula de compilação executada novamente e arquivo verificado antes dos experimentos.
"O comando <code>nvcc</code> não foi encontrado. Como resolver no Colab?"	Identificar que era necessário habilitar GPU no ambiente de execução.	Ambiente ajustado e confirmado com <code>nvidia-smi</code> .
"Revisa para mim a ortografia do meu texto."	Organizar o texto do relatório e concerta a ortografia	Texto revisado pelos autores para ver se não adicionou coisas a mais sem a nossa validação.

8 CONCLUSÃO

O trabalho permitiu comparar diferentes estratégias de otimização para multiplicação de matrizes. A versão sequencial ingênua serviu como referência de corretude, mas apresentou desempenho baixo em comparação com as versões otimizadas. A transposição da matriz B melhorou parte da localidade de memória, mas a técnica que produziu os melhores resultados em CPU foi a blocagem.

Nos testes em CPU, a versão bloqueada com bloco 64 foi a mais eficiente entre as configurações avaliadas, atingindo 12,968695 GFLOP/s em $N = 512$. As versões OpenMP não apresentaram escalabilidade linear, principalmente porque o ambiente possuía apenas dois processadores lógicos disponíveis. Isso mostrou que o ganho de paralelismo depende diretamente do hardware e que o aumento do número de *threads* pode gerar sobrecarga quando há poucos recursos físicos.

Nos testes em GPU, a versão CUDA *tiled* superou a versão CUDA direta em todos os tamanhos testados. O uso de memória compartilhada reduziu acessos à memória global e aumentou o desempenho do kernel. Para $N = 512$, a versão *tiled* atingiu 835,65 GFLOP/s de kernel, enquanto a versão direta atingiu 551,07 GFLOP/s. Para $N = 1024$, a versão *tiled* reduziu o tempo total de 9,204 ms para 7,483 ms.

De forma geral, os resultados mostram que otimizações de memória são fundamentais para GEMM. Em CPU, a blocagem foi decisiva para explorar melhor a cache. Em GPU, o uso de *tiling* e memória compartilhada foi essencial para melhorar o desempenho. A comparação também mostrou que, embora a GPU possa ser muito mais rápida no cálculo, o tempo total da aplicação deve considerar as transferências entre CPU e GPU.

REPOSITÓRIO DO PROJETO

O código-fonte completo e o notebook preenchido estão disponibilizados no seguinte repositório:

[GitHub](#)

REFERÊNCIAS

NETLIB. **BLAS: Basic Linear Algebra Subprograms**. 2026. Acesso em: 25 jun. 2026. Disponível em: <https://www.netlib.org/blas/>.

NVIDIA CORPORATION. **CUDA C++ Programming Guide**. [S.l.], 2026. Acesso em: 25 jun. 2026. Disponível em: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.

OPENMP ARCHITECTURE REVIEW BOARD. **OpenMP Application Programming Interface Version 5.2**. [S.l.], 2021. Acesso em: 25 jun. 2026. Disponível em: <https://www.openmp.org/specifications/>.